# TP - Reductions: the Minimum of an Array

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Part 1

## The exercise: one program, five versions of a reduction

A **reduction** combines the $n$ elements of an array into one value – a sum, a minimum, a dot product. It reads every element once and does one operation per element: its speed is the **bandwidth** it gets from global memory, and the lecture built the sum of an array in four steps, each one closer to that bandwidth. This lab does the same for the **minimum** of an array of $n$ floats, in one program, **find-minimum.cu**, with five versions of the same reduction to write, in order. The lecture's sum becomes a minimum by replacing `+` with `fminf` and the identity 0 with `FLT_MAX` (from `<cfloat>`); everything else carries over.

The host code is given: it fills `a` with values between 0 and 999, hides a single $-1$ at a random position, launches every version, checks that it finds $-1$, and prints the time and the effective bandwidth, $4n$ bytes divided by the time it takes to get the minimum into a CPU variable (the kernel, the copy of the partials, the CPU finish). A **read-only** kernel is given too: it reads `a` once with coalesced loads and computes nothing, and its bandwidth is the *ceiling* that the versions can approach, not pass. The skeleton compiles and runs from the start and prints `WRONG` for every version until it is written. For each version, write the kernel's body following the comment block above it, and set its `dim3` grid and block dimensions in `main()` (as given, every version runs with 1 block of 1 thread). Answer the questions of each exercise in the comment block above the kernel, and put the timing table (Ex. 5b) in a comment at the top of the file.

Every block has `BS` $= 256$ threads (8 warps), a power of two: the tree halves the number of busy threads at every step. The array size is the constant `n` at the top of `main()`, 16 000 001 floats (61 MB), *not* a multiple of `BS`: the threads past the end of `a` must be guarded, and what they put in the tree must not change the minimum. `G` $= 1024$ is the number of blocks of versions 3 to 5. Compile and run with `nvcc -O2 -arch=native find-minimum.cu -o find-minimum`, or paste the file into `cuda.godbolt.org` (compiler options `-O2`), which runs it on a Tesla T4.

Three rules from the lecture, which the check will not forgive you for forgetting:

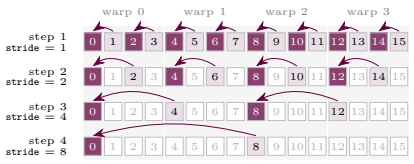
- a thread that reads from shared memory what *another thread* wrote must wait for it: `__syncthreads()` after the load, and after every step of the tree;
- *every* thread of the block must reach *every* `__syncthreads()`: the threads past the end of `a` take part in the tree like the others, with the identity of the minimum in their cell;
- the partials of the blocks are reduced *after* the kernel: a block never waits for another one.

The pictures next to each part use a block of 16 threads in warps of 4, and $G = 4$ blocks; the code uses 256, 32 and 1024.

Part 2

## Version 1: the tree with interleaved pairs

One element per thread, $\lceil n/$`BS`$\rceil$ blocks. Thread $t$ of block $b$ loads `a[b*BS + t]` into `s[t]` (`FLT_MAX` past the end of `a`: the identity of the minimum), then the block runs the **tree** in shared memory: at step `stride` $= 1, 2, 4, \ldots,$ `BS`$/2$, thread $t$ replaces `s[t]` by `fminf(s[t], s[t + stride])` when $t$ is a multiple of $2\cdot$`stride`, and a barrier closes every step. After the last step `s[0]` holds the minimum of the block's elements: thread 0 writes it to `partial[b]`. The CPU takes the minimum of the $\lceil n/$`BS`$\rceil$ partials (written in `main()`).




the tree of one block, 16 threads in warps of 4: dark, the cell thread $t$ writes; pale, the cell it reads

*Ex. 1*

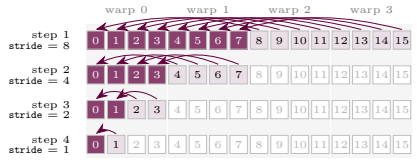
a) Write `min_interleaved` and its launch dimensions, and check that it prints `ok`.

b) With `BS` $= 256$, how many warps execute the comparison at each of the 8 steps? (A warp executes a step as soon as one of its threads is busy.)

Part 3

## Version 2: the tree with contiguous threads

The same load, and the same tree with the busy threads **packed at the front**: at step `stride` $= \texttt{BS}/2, \texttt{BS}/4, \ldots, 1$, the threads $t <$ `stride` replace `s[t]` by `fminf(s[t], s[t + stride])`, a barrier after every step. The threads that are done are the last ones, so whole warps retire step after step, and the % of version 1 is gone.




the same block, the busy threads packed at the front: warps 2 and 3 are done after step 1, warp 1 after step 2

*Ex. 2*

a) Write `min_contiguous` and its launch dimensions.

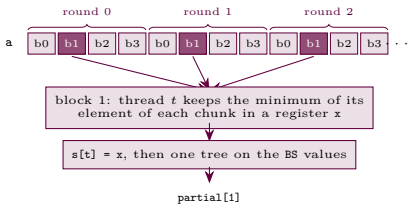
b) How many warps execute each of the 8 steps now? From which step on is a single warp busy?

c) The words a warp reads at a step, `s[t + stride]`: in how many banks?

Part 4

## Version 3: several elements per thread, then the tree

Versions 1 and 2 run a tree of 8 steps and 8 barriers for every `BS` elements, and load nothing meanwhile. This version has only `G` $= 1024$ blocks: thread $t$ of block $b$ walks the array with a **grid-stride loop**, over the elements $b \cdot \texttt{BS} + t$, $b \cdot \texttt{BS} + t + \texttt{G} \cdot \texttt{BS}$, ... (at each turn a warp reads 32 consecutive floats: coalesced), and keeps the minimum of its elements in a register `x`, started at `FLT_MAX`. Then `s[t] = x`, a barrier, and the tree of version 2 on the `BS` values of the block: one tree per block instead of one per `BS` elements, and `G` partials for the CPU instead of $\lceil n/\texttt{BS} \rceil$.




$G = 4$ blocks walk the array in rounds of $G \cdot$`BS` elements; block 1 takes the chunks $1, 5, 9, \ldots$

*Ex. 3*

a) Write `min_multi` and its launch dimensions.

b) How many elements does each thread read, and how many barriers does a block execute? Compare with version 2.

Part 5

## Version 4: the last warp alone

In the tree of version 3, once `stride` $\leq 32$ only warp 0 has busy threads: the 7 other warps have nothing left to do but wait at 6 block barriers. This version stops the loop at `stride` $> 32$ and lets the threads $t < 32$ do the last 6 steps **alone**, without `__syncthreads()`. Inside the warp, put `__syncwarp()` after each step to keep its 32 threads in step (since the Volta generation the threads of a warp may run ahead of each other: this warp barrier is required, not optional). Thread 0 writes `fminf(s[0], s[1])` to `partial[b]`.

| step | stride | warps | barrier |
|---|---|---|---|
| 1 | 128 | 4 | `__syncthreads()` |
| 2 | 64 | 2 | `__syncthreads()` |
| 3 | 32 | 1 | `__syncwarp()` |
| 4 | 16 | 1 | `__syncwarp()` |
| 5 | 8 | 1 | `__syncwarp()` |
| 6 | 4 | 1 | `__syncwarp()` |
| 7 | 2 | 1 | `__syncwarp()` |
| 8 | 1 | 1 | `__syncwarp()` |

the busy threads are `t < stride`; steps 3–8 are warp 0's alone

*Ex. 4*

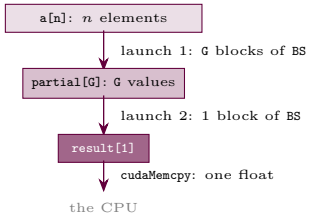
a) Write `min_lastwarp` and its launch dimensions.

b) How many `__syncthreads()` does a block execute now, the one after the load included?

Part 6

## Version 5: the partials reduced by a second launch

Versions 1 to 4 leave their partials to the CPU. The kernel of version 4 works on any array of any length – so on the array of `G` partials too: launched *again*, with a **single block** of `BS` threads, on `partial`, it writes the minimum of the `G` partials to `result[0]`, and the CPU copies one float. There is nothing to write but the dimensions of the second launch in `main()`.




the same kernel twice: on `a`, then on its own partials

*Ex. 5*

a) Set the dimensions of the second launch (`grid5`, `block5`) and check that it prints `ok`. What would go wrong with 2 blocks? With a block of 32 threads?

b) Compare the bandwidth of the five versions with the read-only kernel's: which version reaches it, and what did each step bring? Put the table in a comment at the top of the file.